

Hash Tables

James Anderson

Data Structures and Algorithms

CS 3100/5100

Wright State University

Motivation

- For linked-list and array, linear search (unless elements ordered)
- What if: 8 Digit User IDs valid range 10,000,000 – 99,999,999
- Array with space for 100,000,000
- Use User ID as index
- Sure it's wasteful



Search Time-Complexity

- Store user information in `index == userId`
- Search becomes $O(1)$

Array Index	User Information
123456780	
123456781	
123456782	ID = 123456782 name = Juliet Burke
123456783	
123456784	
123456785	ID = 123456785 name = Hugo Reyes
123456786	
123456787	
123456788	
123456789	ID = 123456789 name = Desmond Hume

Map ADT

- The **Map Abstract Data Type (ADT)** is an unordered collection that stores data in **key-value pairs**, where each unique key is associated with exactly one value. It is also commonly referred to as an **associative array**, **dictionary**, or **symbol table**. Associates (or maps) keys with values
- E.g. Store user information
 - Key: User ID
 - Value: User nameOr
 - Key: User name
 - Value: User ID

Map ADT

Core Properties

- **Key Uniqueness:** Every key in a map must be unique. Adding a new value with an existing key typically overwrites the old value.
- **Value Retrieval:** Values are accessed or modified exclusively via their associated keys rather than a numerical index.
- **Key-Value Separation:** The data types for keys and values do not need to be the same (e.g., a String key can map to an Integer value).

Map ADT Operations

```
Map<int, string> map = {  
    5: "apple",  
    6: "banana",  
    10: "grapefruit"  
}
```

Operation	Description	Example operation	Result
<code>insert(key, value)</code>	Inserts the key-value pair if the key does not exist in the map. (Some Maps also allow updating of the value if key does exist)	<code>map.insert(4, "pear")</code>	<pre>{ 5: "apple", 6: "banana", 10: "grapefruit", 4: "pear" }</pre>
<code>remove(key)</code>	Removes the key and corresponding value	<code>map.remove(10)</code>	<pre>{ 5: "apple", 6: "banana" }</pre>

Map ADT Operations (con't)

```
Map<int, string> map = {  
    5: "apple",  
    6: "banana",  
    10: "grapefruit"  
}
```

Operation	Description	Example operation	Result
<code>get(key)</code>	Returns the value associated with the key, or null if the key does not exist in the Map	<code>map.get(5)</code> <code>map.get(7)</code>	returns "apple" returns null map is unchanged
<code>contains(key)</code>	Returns true if Map contains the key, false otherwise	<code>map.contains(6)</code>	returns true map is unchanged
<code>size()</code>	Returns the number of key-value pairs in the map (aka <code>getLength()</code>)	<code>map.size()</code>	returns 3 map is unchanged

Limited Map ADT Implementation

- Key-value pairs stored in array
- Key used to obtain index
- Issues:
 - Non-integer (unsigned) keys
 - Range of possible keys, not possible to allocate enough space
- Hash code – fixed size, non-negative integer which (attempts to) uniquely identify a key
- Hash function – computes hash code for value
 - Specific to data type

Hash Function

- Attempt to turn value into a unique unsigned integer: hash code
- E.g. integer:
 - Absolute value
- E.g. string
 - Add ASCII code for each character
 - Issue: BAT and TAB have same hash code
 - Add (ASCII code * position) for each character

Common Hash Functions

- Perfect hash function: all keys have unique hash codes
- Good:
 - uniformly distribute items into buckets
 - Use all of the key
- Modulo hash function: $\text{key} \% \text{table.size}()$
 - If keys are 8 digit integers and size is 100, only using part of key
- Mid-square hash function
 - Extract R digits from middle
 - Fast binary implementation
- C++ built in `std::hash<Type>` for primitives, strings

Multiplicative String Hash Function

- ASCII or Unicode value for each character
- Initial value and multiplier often prime

```
int HashMultiplicative(string key, int initialValue = 5381, int hashMultiplier = 33) {  
    int stringHash = initialValue;  
  
    for (char strChar : key) {  
        stringHash = (stringHash * hashMultiplier) + strChar;  
    }  
  
    return stringHash;  
}
```

Hash Table

- Maps (or hashes) items to location in array
- Hash function + data table (array)
- Bucket – array element (index)
- Hash map: Map ADT implementation with hash table
- Key converted to bucket:
 - $\text{hashCode} = \text{hashFunction}(\text{key})$
 - $\text{bucketIndex} = \text{hashCode} \% \text{arrayAllocationSize}$
- hashCode needs to be turned into a valid index
- Valid indexes: 0 to ($\text{arrayAllocationSize} - 1$)

Notation note:
 $N = \text{arrayAllocationSize}$

Hash table data structure

Insert (86, "Paola") into playerNums

Insert (23, "Maria") into playerNums

Insert (42, "Jan") into playerNums

playerNums contains key 23?

playerNums contains key 56?

playerNums:

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

Collisions

- Two different keys map to the same bucket
- E.g. $N = 10$, $\text{hash}(\text{key}) = \text{key} \% N$
 - $\text{hash}(23) = 23 \% 10 = 3$
 - $\text{hash}(83) = 83 \% 10 = 3$
- Collision Resolution Policy
 - **Chaining**: each bucket contains a list
 - **Open addressing**: if a collision occurs, look elsewhere. Where? Elsewhere!

Chaining

- List for each bucket that stores all items that map there
- Get: linear search of list
- Insert: append new item to list
- Remove: remove item from list

Hash table item possible implementation similar to linked list nodes:

ChainingHashTableItem
KeyType key
ValueType value
ChainingHashTableItem* next

ChainingHashTable
Vector<ChainingHashTableItem*> table

Chaining Example

playerNums:

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

Open Addressing

- Each bucket holds one item
- If bucket occupied, look elsewhere
- **Probing:** method for how to look for an open bucket
- **Linear probing:** check the next bucket, then the next...
- Start bucket $\rightarrow$ home position
- Probe $\rightarrow$ home + probe(key, i)
- i is the i^{th} probe iteration

Linear Probing

- $\text{home} = \text{hash}(\text{key}) \% N$
- $\text{probe}(\text{key}, i) = (\text{home} + i) \% N$
- **insert: check home position, if collision, probe starting $i = 1$**
 - If we find an empty bucket, stop and insert
 - If we find the key, stop and return false (no duplicates)
- **search: check home position, if not key probe starting $i = 1$**
 - If we find key, return the value
 - If we reach an empty slot, key not in table
- **remove: check home position, if not key probe starting $i = 1$**
 - If we find key, remove from table
 - If we reach empty slot, key not in table

Linear Probing

Open Addressing (Linear Probing)

A: $h(A)=65 \rightarrow 65\%5=0$ ✓ K: $h(K)=75 \rightarrow 75\%5=0$
F: $h(F)=70 \rightarrow 70\%5=0$ ✓
G: $h(G)=71 \rightarrow 71\%5=1$ ✓



Open Addressing (Linear Probing)

A: $h(A)=65 \rightarrow 65\%5=0$ ✓ K: $h(K)=75 \rightarrow 75\%5=0$
F: $h(F)=70 \rightarrow 70\%5=0$ ✓
G: $h(G)=71 \rightarrow 71\%5=1$ ✓ B: $h(B)=66 \rightarrow 66\%5=1$



If the current array overflows, then create a new array and rehash the keys.

Open Addressing (Linear Probing)

A: $h(A)=65 \rightarrow 65\%5=0$ ✓ K: $h(K)=75 \rightarrow 75\%5=0$
F: $h(F)=70 \rightarrow 70\%5=0$ ✓ B: $h(B)=66 \rightarrow 66\%5=1$
G: $h(G)=71 \rightarrow 71\%5=1$ ✓



Empty Bucket Types

- Issue with remove and subsequent inserts/searches:
- $N = 10, \text{hash}(\text{key}) = \text{key} \% N$
- `insert(34), insert(64), remove(34)`
- `search(64)`
- Denote three bucket types:
 - Empty since start (ESS)
 - Slot has never been used
 - If you hit this during search → stop
 - Safe to conclude: key is not in table
 - Empty after removal (EAR)
 - Slot used to have something, but was deleted
 - During search → do NOT stop
 - Keep probing
 - Normal
 - Contains a valid key

Clustering

- Probe sequence: home position, first probe location, second...
- Linear probing: probe sequence for ($N = 10$)
 - home = 2: 2, 3, 4, 5, 6, 7, 8, 9, 0, 1
 - home = 7: 7, 8, 9, 0, 1, 2, 3, 4, 5, 6
- Sequences converge at some point and never diverge
- Increased probability of collisions, then more collisions
- E.g. one item in table, probability of next having same home = $1/N$, two items

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

Quadratic probing

- Attempt to solve clustering from Linear Probing
- h = home = hash code
- Bucket index = $(h + c_1 * i + c_2 * i^2) \% N$
- c_1 and c_2 constants chosen (carefully) by programmer
- E.g. $c_1 = 3, c_2 = 8, N = 10$
 - H = 3, probes: 4, 1, 4, 3, 8, 9, 6, 9, 8, 3
 - H = 7, probes: 8, 5, 8, 7, 2, 3, 0, 3, 2, 7
- Many c 's are prime

Probe	$h = 3$	$h = 7$
1	4	8
2	1	5
3	4	8
4	3	7
5	8	2
6	9	3
7	6	0
8	9	3
9	8	2
10	3	7

Pseudo-random Probing

- Quadratic probing: doesn't guarantee we check all buckets
- Linear probing does: $\text{home} + 1, \text{home} + 2, \text{home} + 3 \dots \text{home} + N - 1$
- Randomize offsets from home:
- $\text{home} + 4, \text{home} + 7, \text{home} + 2, \text{home} + N - 1, \dots, \text{home} + 1$
- Need to use same randomized sequence – pre-generate array
- Offsets array: size = $N - 1$, contains 1, 2, 3, ... $N-1$ shuffled
- E.g. $N = 10$, offsets array:

$\text{Probe}(\text{key}, i) = (\text{home} + \text{offsets}[i]) \% N$

4	7	2	9	1	8	6	3	5
---	---	---	---	---	---	---	---	---

Pseudo-Random Probing

4	7	2	9	1	8	6	3	5
---	---	---	---	---	---	---	---	---

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

Clustering (again)

- Quadratic and Pseudo-random probing: different home positions, probe sequences diverge. Great!
- *Different* keys with *same* home positions?
- **Primary Clustering:** keys that map to *different* home positions follow the same probe sequence
- **Secondary Clustering:** keys that map to the *same* home position follow the same probe sequence

Double Hashing

- So far probe function only used the probe iteration
- What if probe used the iteration and the key?
- Different keys should generate different probe sequences
- Two hash functions $h_1(key)$ and $h_2(key)$
- Bucket index: $(h_1(key) + i * h_2(key)) \% N$
- If $i=0$, index = $(h_1(key)) \% N$ = home position
- Probe function = $i * h_2(key)$

Hash Table Resizing

- If we keep table half full, on average at most 2 collisions
- Commonly increase to next prime $\geq 2 * N$
- Can we just copy from old array into new array?
- Because N changes, we need to re-insert from old into new
- Thresholds:
 - Load factor
 - Number of collisions during an insert
 - Size of bucket's linked list